CASE-BASED REASONING SYSTEM DOCUMENTATION

Geofrey Miguel Erlano & Lance Kirsten Buenaventura

CS0065-AN41
October 3, 2026

Disaster Response Decision Support System (DSS)

Our CBR system stores historical disaster cases locally in a JSON database (disaster_memory.json). The case base contains highly detailed, real-world scenarios representing Typhoons, Earthquakes, and Floods across three primary Philippine regions (Luzon, Visayas, and Mindanao). Each case is structured as a dictionary containing a historical_context, a problem description (disaster type, intensity, location, and proximity hazard), and a complex solution (evacuation strategy, shelter capacity, categorized resource allocation, and friendly community tips). Furthermore, our system dynamically expands its case base by fetching live earthquake data from the USGS API, combining local memory with real-time global internet data.

Explanation of Similarity Method (Retrieve Phase)

We implemented a Weighted Scoring Algorithm to calculate similarity. The system evaluates the new disaster against all past cases in the database by assigning points to matching attributes:

- Disaster Type Match: +5 points (Highest priority to ensure earthquakes aren't matched with floods).
- Location Match: +3 points (If the historical case shares the same region, e.g., Visayas).
- Hazard Match: +2 points (If both the past and new problem share a proximity hazard, like a coastline or mountain).
- Intensity Similarity: Up to +5 points. This is calculated using the formula 5 - absolute difference in intensity. This ensures that a Magnitude 7.0 gets a higher score matching with a past 7.2 than with a past 5.0.
- The case with the highest cumulative score is retrieved as the most relevant past experience.

Adaptation Logic (Reuse and Revise Phase)

Because no two disasters are exactly alike, the system adapts the retrieved solution to fit the new context through **Rule-Based Adaptation**:

- **Intensity Scaling:** The system adapts shelter capacity using a numeric calculation. When the new disaster's intensity exceeds the retrieved case's intensity by more than 1.0, the program increases the stored capacity by 50% and displays the original and adjusted values. Before retention, the user can accept the recommendation or revise the capacity. The system then saves the reviewed solution with the new problem description in the JSON case base. This makes the reuse, revision, and retention stages visible during execution.
- **Proximity Hazard Protocols:** The AI injects specific protocols depending on the environment. If the user indicates they are near a "coastline," it enforces a 5km no-entry zone, deploys Coast Guard vessels, and warns of storm surges. If near a "mountain," it warns of landslides, evaluates soil liquefaction, and allocates heavy payloaders. It also dynamically

generates tailored "friendly tips" for the community based on these exact variables.

Screenshots of Program Execution

Figure 1. New Problem and Case Retrieval
The system retrieves a similar typhoon case with a similarity score of 13.00.

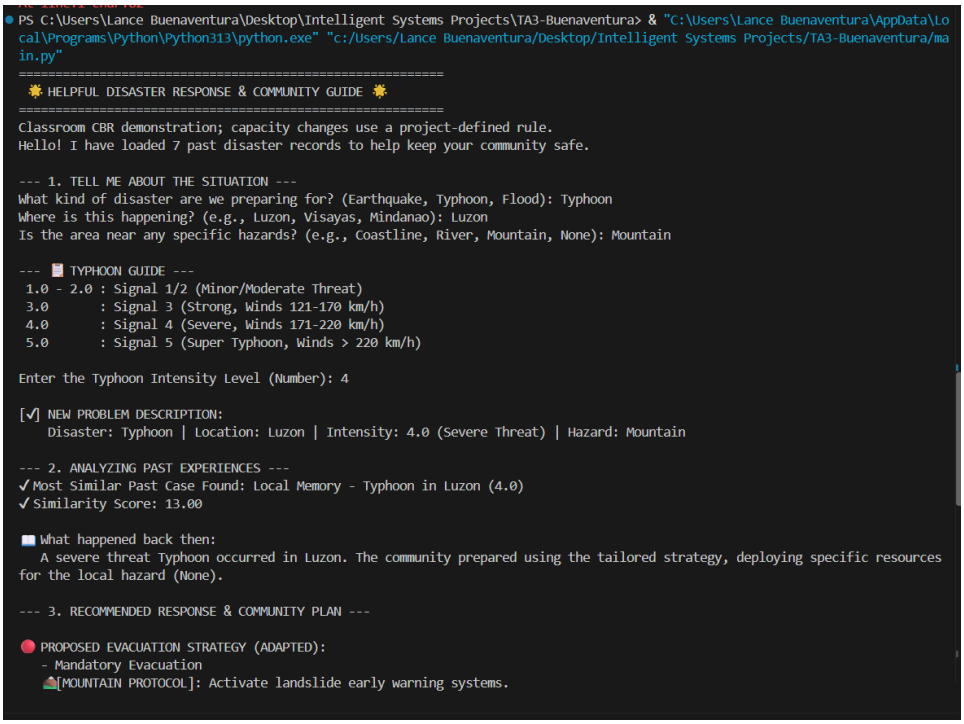


Figure 2. Shelter Capacity Revision and Saving
The user revises shelter capacity from 5,000 to 12,500 beds, and the system saves the reviewed plan.

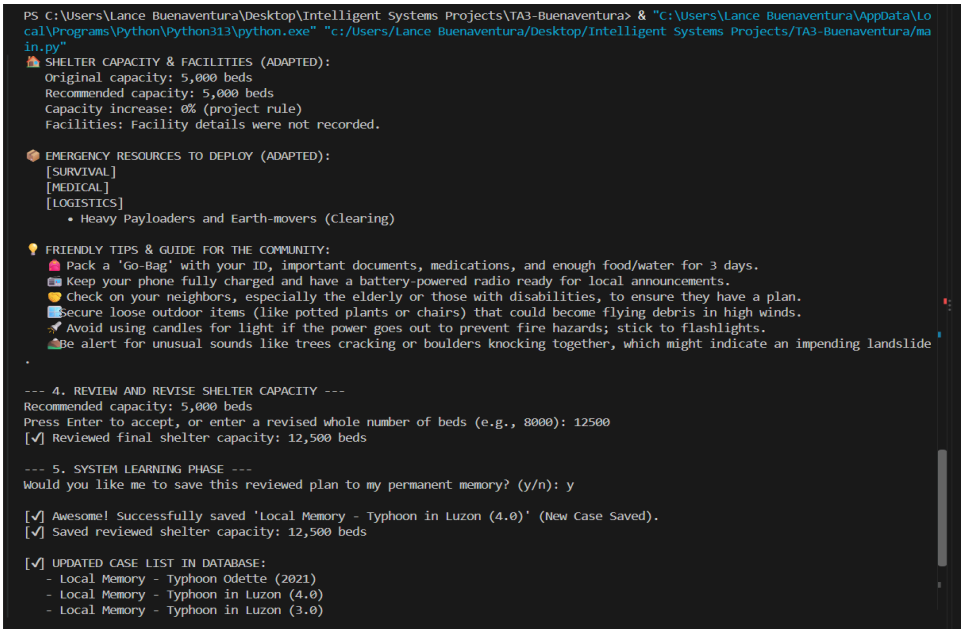
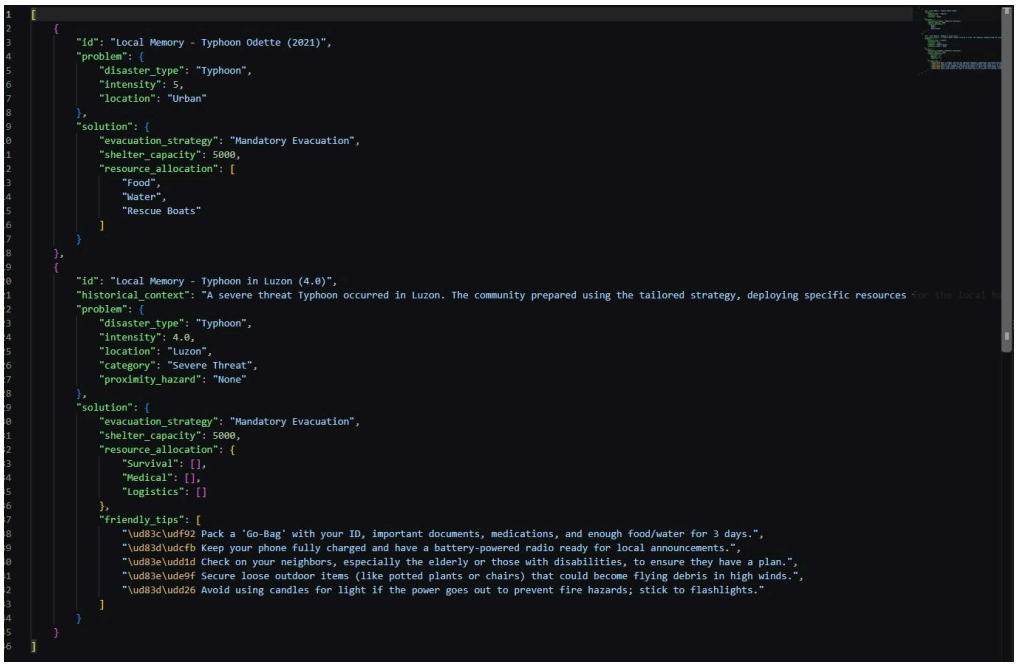


Figure 3. Updated Case Base
The updated case list includes the newly saved scenario.




```
[
    {
        "id": "Local Memory - Typhoon Odette (2021)",
        "problem": {
            "disaster_type": "Typhoon",
            "intensity": 5,
            "location": "Urban"
        },
        "solution": {
            "evacuation_strategy": "Mandatory Evacuation",
            "shelter_capacity": 5000,
            "resource_allocation": [
                "Food",
                "Water",
                "Rescue Boats"
            ]
        }
    },
    {
        "id": "Local Memory - Typhoon in Luzon (4.0)",
        "historical_context": "A severe threat Typhoon occurred in Luzon. The community prepared using the tailored strategy, deploying specific resources for the local ha
        "problem": {
            "disaster_type": "Typhoon",
            "intensity": 4.0,
            "location": "Luzon",
            "category": "Severe Threat",
            "proximity_hazard": "None"
        },
        "solution": {
            "evacuation_strategy": "Mandatory Evacuation",
            "shelter_capacity": 5000,
            "resource_allocation": {
                "Survival": [],
                "Medical": [],
                "Logistics": []
            },
            "friendly_tips": [
                "\ud83c\udf92 Pack a 'Go-Bag' with your ID, important documents, medications, and enough food/water for 3 days.",
                "\ud83d\udcfb Keep your phone fully charged and have a battery-powered radio ready for local announcements.",
                "\ud83e\udd1d Check on your neighbors, especially the elderly or those with disabilities, to ensure they have a plan.",
                "\ude9f Secure loose outdoor items (like potted plants or chairs) that could become flying debris in high winds.",
                "\ud83d\udd26 Avoid using candles for light if the power goes out to prevent fire hazards; stick to flashlights."
            ]
        }
    }
]
```


Pair Reflection

- **What each partner contributed**
  - Erlano spearheaded the core Python architecture, establishing the interactive command-line interface and the main CBR control flow. They engineered the weighted similarity assessment algorithm, which mathematically ranks past cases by assigning specific point values to matching disaster types, regional locations, and proximity hazards, while calculating exact intensity differentials. Furthermore, they integrated the requests library to connect the system to the live USGS Earthquake API, enabling the program to fetch, parse, and utilize real-world GeoJSON disaster records on the fly.

  - Lance constructed the local JSON knowledge base, carefully designing the dictionary structures to hold historical contexts, problem states, and categorized solutions (Survival, Medical, Logistics). They developed the complex rule-based adaptation logic, programming the AI to automatically scale shelter capacities based on intensity and inject specific environmental hazard protocols—such as deploying Coast Guard vessels for coastal threats or heavy payloaders for mountain landslides. Finally, they conducted rigorous system testing, utilizing defensive programming techniques to ensure the program handled varying data types without crashing during the retention phase.

· **What challenges you encountered**

- Our main challenge was fine-tuning the weighted scoring system so the AI wouldn't accidentally match an earthquake with a typhoon just because they shared the same intensity level. Additionally, we had to implement deep-copy functions to ensure the AI could adapt complex resource lists without permanently overwriting or corrupting the original historical cases.

· **What you learned about CBR**

- This project taught us that Case-Based Reasoning is incredibly efficient for dynamic environments like disaster response because it mimics how human experts adapt past knowledge to fit new constraints. Most importantly, we saw firsthand the power of the "Retain" phase; by permanently saving our adapted solutions to a local file, the system actively learns and improves its own reasoning without requiring us to write new code.